

```
Start here x d detection.c x
1  #include <stdio.h>
2  int main()
3  {
4      int n, m, i, j, k;
5      printf("Enter number of processes: ");
6      scanf("%d", &n);
7      printf("Enter number of resource types: ");
8      scanf("%d", &m);
9      int allocation[n][m], request[n][m];
10     int available[m];
11     printf("\nEnter Allocation Matrix:\n");
12     for(i = 0; i < n; i++)
13     {
14         for(j = 0; j < m; j++)
15         {
16             scanf("%d", &allocation[i][j]);
17         }
18     }
19     printf("\nEnter Request Matrix:\n");
20     for(i = 0; i < n; i++)
21     {
22         for(j = 0; j < m; j++)
23         {
24             scanf("%d", &request[i][j]);
25         }
26     }
27     printf("\nEnter Available Vector:\n");
28     for(i = 0; i < m; i++)
29     {
30         scanf("%d", &available[i]);
31     }
32     int work[m], finish[n];
33     for(i = 0; i < m; i++)
34     {
35         work[i] = available[i];
36     }
37     for(i = 0; i < n; i++)
38     {
39         int sum = 0;
40         for(j = 0; j < m; j++)
41         {
42             sum += allocation[i][j];
43         }
44         if(sum == 0)
45             finish[i] = 1;
46         else
47             finish[i] = 0;
48     }
49     int found;
```

```

47         finish[i] = 0;
48     }
49     int found;
50     do
51     {
52         found = 0;
53         for(i = 0; i < n; i++)
54         {
55             if(finish[i] == 0)
56             {
57                 int possible = 1;
58                 for(j = 0; j < m; j++)
59                 {
60                     if(request[i][j] > work[j])
61                     {
62                         possible = 0;
63                         break;
64                     }
65                 }
66                 if(possible)
67                 {
68                     for(k = 0; k < m; k++)
69                     {
70                         work[k] += allocation[i][k];
71                     }
72                     finish[i] = 1;
73                     found = 1;
74                 }
75             }
76         }
77     } while(found);
78     int deadlock = 0;
79     printf("\n");
80     for(i = 0; i < n; i++)
81     {
82         if(finish[i] == 0)
83         {
84             deadlock = 1;
85             printf("Process P%d is deadlocked.\n", i);
86         }
87     }
88     if(deadlock == 0)
89     {
90         printf("No Deadlock Detected.\n");
91     }
92     return 0;
93 }
94

```

Enter number of processes: 5
Enter number of resource types: 3

Enter Allocation Matrix:
0
1
0
2
0
0
3
0
3
2
1
1
0
0
2

Enter Request Matrix:
0
0
0
2
0
2
0
0
1
1
0
0
0
0
2

Enter Available Vector:
0
0
0

Process P1 is deadlocked.
Process P2 is deadlocked.
Process P3 is deadlocked.
Process P4 is deadlocked.

Process returned 0 (0x0) execution time : 61.474 s
Press any key to continue.
|